

Códigos de Control de Errores

(Unidad 3)

Guerra Jorge, Pesce Agustín, Villar Federico

23 de julio de 2026

Resumen

Esta unidad presenta la primera familia formal de códigos correctores del curso, y aplica directamente la estructura algebraica desarrollada en la unidad anterior. Un código de bloque lineal (n, k) se define como un subespacio de dimensión k del espacio de las n -tuplas binarias, lo que reduce la codificación a un producto matriz-vector y hace tratable un problema que, sobre un subconjunto arbitrario de palabras, sería de complejidad exponencial. Se estudian la matriz generadora G y su forma sistemática, la matriz de paridad H y el código dual, y el síndrome, cuya propiedad central es depender únicamente del patrón de error y no de la palabra transmitida. A partir del peso y la distancia de Hamming se deducen la distancia mínima del código y las capacidades de detección y corrección que de ella se derivan, junto con las cotas de Singleton y de Hamming. Se presentan luego los dos esquemas clásicos de decodificación —el standard array y la decodificación por síndrome—, se analiza la performance sobre el canal binario simétrico y se introducen las nociones de umbral y ganancia de codificación. La unidad cierra con tres familias particulares —paridad simple, repetición y códigos auto-duales— que ilustran los extremos del compromiso entre tasa y capacidad de corrección. El código de Hamming $(7, 4)$ se emplea como ejemplo transversal, y los scripts de Python que acompañan al apunte permiten reproducir todas sus tablas y figuras.

1. De la estructura algebraica al código

La unidad anterior terminó con una observación que ahora se convierte en punto de partida. Se vio que el conjunto V_n de todas las n -tuplas binarias es un espacio vectorial sobre $GF(2)$, que un subespacio de dimensión k contiene exactamente 2^k vectores, y que a todo subespacio le corresponde un espacio dual cuya dimensión es $n - k$. Esos tres hechos, aparentemente inocuos, contienen ya la definición completa de un código de bloque lineal, su codificador y su detector de errores.

La idea es la siguiente. De las 2^n palabras posibles de longitud n que pueden circular por el canal, sólo 2^k se declaran *válidas*. Esas 2^k palabras alcanzan para transportar los 2^k mensajes distintos de k bits, y las $2^n - 2^k$ restantes quedan como palabras “prohibidas”. Cuando el receptor observa una palabra prohibida, sabe con certeza que hubo errores; y si el conjunto de palabras válidas está bien elegido, puede además deducir cuál era la palabra transmitida más probable. Toda la teoría de los códigos de bloque consiste en elegir bien ese subconjunto.

La restricción que se impone en esta unidad, y que da nombre a la familia, es que el subconjunto elegido sea un *subespacio* y no un subconjunto cualquiera. La motivación es puramente práctica. Un subconjunto arbitrario de 2^k palabras de longitud n exigiría, para codificar, una tabla de 2^k entradas de n bits, y para decodificar, una búsqueda sobre esa tabla. Para el código (255, 239) del trabajo práctico final eso significa 2^{239} entradas, un número sin sentido físico. Si, en cambio, el conjunto es un subespacio, queda determinado por apenas k vectores base, la codificación se reduce a un producto matriz-vector, y la verificación a otro producto. La linealidad no es una hipótesis matemática cómoda: es lo que vuelve al problema implementable.

El recorrido de la unidad es el siguiente. Primero se define el código a partir de su matriz generadora G y se introduce la forma sistemática. Luego aparece la matriz de paridad H , dual de la anterior, y con ella el *síndrome*, que es el objeto central de toda la decodificación algebraica. A continuación se estudian el peso y la distancia mínima, de los cuales se deducen las capacidades de detección y corrección del código. Después se presentan los dos esquemas clásicos de decodificación, el *standard array* y la decodificación por síndrome, y se analiza la performance del código sobre un canal binario simétrico. La unidad cierra con tres familias particulares que ilustran los extremos del compromiso entre redundancia y capacidad de corrección.

2. Definición y matriz generadora

2.1. Definición

Un *código de bloque lineal* (n, k) sobre $GF(2)$ es un subespacio de dimensión k del espacio vectorial V_n de todas las n -tuplas binarias. Sus 2^k elementos se denominan *palabras de código* (*codewords*); n es la *longitud de bloque* y k la *dimensión* o cantidad de *símbolos de información*. La diferencia $n - k$ es la cantidad de *símbolos de redundancia* o de *paridad*, y el cociente

$$R = \frac{k}{n}$$

es la *tasa de código* (*code rate*), que mide qué fracción del ancho de banda transporta información útil. Una tasa cercana a 1 significa poca redundancia; una tasa baja, mucha protección a costa de eficiencia.

De la definición como subespacio se desprenden de inmediato dos propiedades que se usarán sin cesar:

1. La **palabra nula** $\mathbf{0} = (0, 0, \dots, 0)$ pertenece siempre al código, cualquiera sea éste.
2. La **suma de dos palabras de código es otra palabra de código**. En consecuencia, el conjunto de palabras “se ve igual” desde cualquiera de ellas, hecho que se explotará más adelante para reducir el análisis de errores al caso en que se transmite la palabra nula.

Conviene notar que la segunda propiedad no es gratuita: existen códigos no lineales, y algunos son excelentes, pero pierden justamente la estructura que hace tratable la decodificación.

2.2. Matriz generadora

Como el código C es un subespacio de dimensión k , admite una base $\mathbf{g}_0, \mathbf{g}_1, \dots, \mathbf{g}_{k-1}$ de k vectores linealmente independientes de V_n . Disponiéndolos como filas se obtiene la *matriz generadora* (*generator matrix*) del código:

$$G = \begin{bmatrix} \mathbf{g}_0 \\ \mathbf{g}_1 \\ \vdots \\ \mathbf{g}_{k-1} \end{bmatrix} = \begin{bmatrix} g_{00} & g_{01} & \cdots & g_{0,n-1} \\ g_{10} & g_{11} & \cdots & g_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ g_{k-1,0} & g_{k-1,1} & \cdots & g_{k-1,n-1} \end{bmatrix} \quad (1)$$

Toda palabra de código es una combinación lineal de las filas de G . Si el mensaje a transmitir es el vector $\mathbf{u} = (u_0, u_1, \dots, u_{k-1})$ de k bits, la palabra de código asociada es

$$\mathbf{v} = \mathbf{u} \cdot G = u_0 \mathbf{g}_0 + u_1 \mathbf{g}_1 + \cdots + u_{k-1} \mathbf{g}_{k-1} \quad (2)$$

Ésta es la operación de codificación completa, y su costo es notable por lo bajo: el codificador no necesita almacenar las 2^k palabras de código sino apenas las $k \cdot n$ entradas de G . Para el (255, 239) mencionado antes, se pasa de 2^{239} entradas a poco más de 60 000 bits, y en la práctica ni siquiera hace falta almacenar G , porque los códigos que se usan tienen estructura adicional (cíclica) que permite generarla al vuelo con un registro de desplazamiento.

Obsérvese que la base de un subespacio no es única: distintas elecciones de G producen el *mismo* conjunto de palabras de código, asociando cada mensaje a una palabra distinta. La correspondencia mensaje-palabra cambia, el código no.

2.3. Forma sistemática

Entre todas las matrices generadoras posibles de un mismo código, hay una particularmente cómoda. Se dice que G está en *forma sistemática* (*systematic form*, o forma escalonada reducida) cuando se escribe como

$$G = \begin{bmatrix} I_k & P \end{bmatrix} \quad (3)$$

donde I_k es la matriz identidad de $k \times k$ y P es una matriz de $k \times (n - k)$. Como se vio en la unidad anterior, toda matriz de rango k puede llevarse a esta forma mediante operaciones elementales de fila y, si hace falta, permutaciones de columnas.

Desarrollando (2) con esta G , la palabra de código resulta

$$\mathbf{v} = (\underbrace{u_0, u_1, \dots, u_{k-1}}_{\text{mensaje}}, \underbrace{p_0, p_1, \dots, p_{n-k-1}}_{\text{paridad}}),$$

es decir, el mensaje aparece *intacto* en las primeras k posiciones, seguido de $n - k$ símbolos de paridad dados por

$$p_j = u_0 p_{0j} + u_1 p_{1j} + \cdots + u_{k-1} p_{k-1,j}, \quad j = 0, 1, \dots, n - k - 1.$$

Cada símbolo de paridad es, entonces, la suma módulo 2 de un subconjunto de los

bits de mensaje, y se implementa con un árbol de compuertas XOR. La ventaja operativa es evidente: en recepción, si no hubo errores, los bits de información se leen directamente, sin necesidad de invertir ninguna transformación.

La convención de ubicar el mensaje al principio y la paridad al final es arbitraria; buena parte de la bibliografía, incluido Lin & Costello, adopta el orden inverso, $G = \begin{bmatrix} P & I_k \end{bmatrix}$. Ambas producen códigos equivalentes, con idénticas propiedades de distancia. En este apunte se usa consistentemente la forma (3).

2.4. Ejemplo: el código (7, 4)

A lo largo de toda la unidad se usará como ejemplo el código (7, 4) generado por

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} I_4 & P \end{bmatrix} \quad (4)$$

Sus tres ecuaciones de paridad son, leyendo las columnas de P ,

$$p_0 = u_0 + u_2 + u_3,$$

$$p_1 = u_0 + u_1 + u_2,$$

$$p_2 = u_1 + u_2 + u_3.$$

Con $n = 7$ y $k = 4$, la tasa es $R = 4/7 \approx 0,571$: algo más de la mitad del flujo transmitido es información. La Tabla 1 lista las 16 palabras de código.

Este código no es cualquier código de siete bits: es el código de Hamming (7, 4), el primer código corrector de errores publicado, y se estudiará como miembro de una familia en la unidad siguiente. Aquí se lo usa simplemente como banco de pruebas de los conceptos generales.

Tabla 1: Las 16 palabras del código (7, 4) generado por (4), con su peso de Hamming. Los cuatro primeros bits son el mensaje; los tres últimos, la paridad.

Mensaje u	Palabra v	Mensaje u	Palabra v
0000	0000 000 ($w = 0$)	0110	0110 100 ($w = 3$)
1000	1000 110 ($w = 3$)	0101	0101 110 ($w = 4$)
0100	0100 011 ($w = 3$)	0011	0011 010 ($w = 3$)
0010	0010 111 ($w = 4$)	1110	1110 010 ($w = 4$)
0001	0001 101 ($w = 3$)	1101	1101 000 ($w = 3$)
1100	1100 101 ($w = 4$)	1011	1011 100 ($w = 4$)
1010	1010 001 ($w = 3$)	0111	0111 001 ($w = 4$)
1001	1001 011 ($w = 4$)	1111	1111 111 ($w = 7$)

3. Matriz de paridad y código dual

3.1. Definición

La matriz generadora describe el código “desde adentro”: dice cómo construir sus palabras. Existe una descripción complementaria, que lo describe “desde afuera”, mediante las condiciones que una palabra debe satisfacer para pertenecer a él. Esa descripción es la que se usa en recepción, y es la que da origen al síndrome.

Sea C un código (n, k) , subespacio de V_n . Su espacio dual C_d , según lo visto en la unidad anterior, es el subespacio de dimensión $n - k$ formado por todos los vectores ortogonales a todas las palabras de C . Cualquier matriz H de $(n - k) \times n$ cuyas filas constituyan una base de C_d se denomina *matriz de paridad* (*parity-check matrix*) del código, y satisface

$$G \cdot H^T = \mathbf{0} \quad (5)$$

De aquí se obtiene el criterio de pertenencia que buscábamos. Un vector $\mathbf{v}$ de V_n es palabra de código **si y sólo si**

$$\mathbf{v} \cdot H^T = \mathbf{0} \quad (6)$$

La implicación directa es inmediata: si $\mathbf{v} = \mathbf{u}G$, entonces $\mathbf{v}H^T = \mathbf{u}GH^T = \mathbf{0}$. La recíproca se sigue de que el espacio nulo de H tiene dimensión $n - (n - k) = k$ y contiene a C , que también tiene dimensión k : ambos subespacios coinciden.

Cada una de las $n - k$ filas de H expresa entonces una *ecuación de control de paridad* (*parity-check equation*) que toda palabra de código debe satisfacer. El código queda descripto, indistintamente, como el espacio fila de G o como el espacio nulo de H .

3.2. Obtención de H a partir de G

Cuando G está en forma sistemática, H se escribe de inmediato. Si $G = \begin{bmatrix} I_k & P \end{bmatrix}$, entonces

$$H = \begin{bmatrix} P^T & I_{n-k} \end{bmatrix} \quad (7)$$

En efecto, $GH^T = I_k P + P I_{n-k} = P + P = \mathbf{0}$, donde la última igualdad usa que en característica 2 todo elemento es su propio inverso aditivo. Las $n - k$ filas de H son linealmente independientes por la presencia del bloque identidad, de modo que H tiene rango $n - k$ y es efectivamente una matriz de paridad.

Para el código $(7, 4)$ del ejemplo, transponiendo el bloque P de (4) se obtiene

$$H = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix} \quad (8)$$

Vale la pena mirar las *columnas* de esta matriz, porque en ellas está codificada, literalmente, toda la capacidad de corrección del código. Las siete columnas son

$$\mathbf{h}_0 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \mathbf{h}_1 = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}, \mathbf{h}_2 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, \mathbf{h}_3 = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}, \mathbf{h}_4 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \mathbf{h}_5 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \mathbf{h}_6 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix},$$

es decir, exactamente los siete vectores no nulos de V_3 , cada uno una sola vez. Este hecho, que a esta altura parece una casualidad de la construcción, es la definición misma de los códigos de Hamming y la razón de su capacidad de corrección.

3.3. El código dual

El espacio dual C_d es a su vez un código de bloque lineal, de parámetros $(n, n - k)$, denominado *código dual* de C . Su matriz generadora es H y su matriz de paridad es G : los roles se intercambian. Esta simetría no es una curiosidad formal. Se verá al final de la unidad que el código de repetición y el código de paridad simple son duales entre sí, y en la unidad de códigos LDPC la matriz H pasa a ser el objeto de diseño primario, relegando a G a un papel secundario.

4. El síndrome

4.1. Definición y propiedad fundamental

Supóngase que se transmite la palabra de código $\mathbf{v}$ y que el canal introduce errores. La palabra recibida $\mathbf{r}$ difiere de $\mathbf{v}$ en las posiciones erróneas, lo que se modela sumando un *patrón de error* (*error pattern*) $\mathbf{e}$:

$$\mathbf{r} = \mathbf{v} + \mathbf{e} \quad (9)$$

El vector $\mathbf{e}$ tiene un 1 en cada posición donde el canal invirtió el bit y un 0 en las demás. Nótese que este modelo aditivo sobre $GF(2)$ describe exactamente el canal binario simétrico, y que la tarea del decodificador es, en el fondo, estimar $\mathbf{e}$.

Se define el *síndrome* (*syndrome*) de la palabra recibida como el vector de $n - k$ componentes

$$\mathbf{s} = \mathbf{r} \cdot H^T = (s_0, s_1, \dots, s_{n-k-1}) \quad (10)$$

El nombre proviene de la medicina: el síndrome es el conjunto de síntomas observables a partir de los cuales se diagnostica la enfermedad, sin observarla directamente. Aquí se calcula a partir de lo recibido y permite diagnosticar el error sin conocer lo transmitido.

La propiedad central es la siguiente. Sustituyendo (9) en (10) y usando (6),

$$\mathbf{s} = (\mathbf{v} + \mathbf{e})H^T = \mathbf{v}H^T + \mathbf{e}H^T = \mathbf{e} \cdot H^T \quad (11)$$

Es decir: **el síndrome no depende de la palabra transmitida, sino únicamente del patrón de error**. Esto es lo que vuelve viable toda la decodificación algebraica. El receptor no sabe qué se transmitió —eso es justamente lo que quiere averiguar—, pero puede calcular una cantidad que depende sólo de lo que el canal hizo. De (11) se sigue de inmediato el criterio de detección:

- Si $\mathbf{s} = \mathbf{0}$, la palabra recibida es una palabra de código y se la acepta.
- Si $\mathbf{s} \neq \mathbf{0}$, hubo errores con certeza.

La asimetría entre ambos casos merece subrayarse. Un síndrome no nulo es prueba concluyente de error; un síndrome nulo **no** es prueba de ausencia de error. Si el patrón de error coincide con una palabra de código no nula, entonces $\mathbf{e}H^T = \mathbf{0}$ y el error pasa completamente inadvertido. Estos son los *errores no detectados* (*undetected errors*), y hay exactamente $2^k - 1$ patrones de error que los producen. Su probabilidad se calcula en la Sección 8.

4.2. El síndrome como suma de columnas

Desarrollando (11) componente a componente, y llamando $\mathbf{h}_i$ a la i -ésima columna de H , se obtiene

$$\mathbf{s}^T = \sum_{i=0}^{n-1} e_i \mathbf{h}_i = \sum_{i \in \mathcal{E}} \mathbf{h}_i \quad (12)$$

donde $\mathcal{E}$ es el conjunto de posiciones erróneas. Dicho en palabras: **el síndrome es la suma de las columnas de H correspondientes a las posiciones en error**. Esta lectura es mucho más útil que la fórmula matricial y conviene fijarla, porque es la que explica todo lo que sigue.

El caso de un único error es inmediato: si el error está en la posición i , el síndrome es directamente la columna $\mathbf{h}_i$. Por lo tanto, **si todas las columnas de H son distintas y no nulas, todo error simple produce un síndrome distinto y queda localizado sin ambigüedad**. Ésa es precisamente la propiedad que se observó en la matriz (8) del ejemplo: sus siete columnas son los siete vectores no nulos de V_3 , de modo que el síndrome *es* la representación binaria del número de posición del bit erróneo, y el decodificador se reduce a un decodificador de 3 a 7 líneas.

Ejemplo. Transmítase la palabra $\mathbf{v} = 1001\,011$ (mensaje $\mathbf{u} = 1001$) y supóngase que el canal invierte el bit de la posición 2, de modo que se recibe $\mathbf{r} = 1011\,011$. El síndrome resulta

$$\mathbf{s} = \mathbf{r}H^T = (1, 1, 1),$$

que coincide con la columna $\mathbf{h}_2$ de (8). El decodificador concluye que el error está en la posición 2, suma $\mathbf{e} = 0010\,000$ a la palabra recibida y recupera $\mathbf{v} = 1001\,011$. Nótese que en ningún momento se comparó $\mathbf{r}$ con las 16 palabras de código.

4.3. La ambigüedad esencial

Hay, sin embargo, un límite que ninguna astucia algorítmica puede sortear, y conviene enunciarlo antes de estudiar los algoritmos de decodificación para no generar expectativas falsas. El síndrome tiene $n - k$ componentes, de modo que existen 2^{n-k} síndromes posibles; los patrones de error, en cambio, son 2^n . Por lo tanto, a cada síndrome le corresponden exactamente

$$\frac{2^n}{2^{n-k}} = 2^k \quad (13)$$

patrones de error distintos. El sistema de ecuaciones $\mathbf{e}H^T = \mathbf{s}$ tiene n incógnitas y sólo $n - k$ ecuaciones: está indeterminado, y sus 2^k soluciones son igualmente compatibles con lo observado.

El decodificador debe entonces *elegir* una de esas 2^k soluciones. La elección no

puede fundarse en el álgebra —todas satisfacen las ecuaciones— sino en la estadística del canal. Ése es el contenido de la Sección 7.

5. Peso, distancia y distancia mínima

5.1. Definiciones

El *peso de Hamming* (*Hamming weight*) de un vector $\mathbf{v}$ de V_n , que se denota $w(\mathbf{v})$, es la cantidad de componentes no nulas del vector. La *distancia de Hamming* entre dos vectores $\mathbf{u}$ y $\mathbf{v}$, denotada $d(\mathbf{u}, \mathbf{v})$, es la cantidad de posiciones en las que difieren.

Ambos conceptos están ligados por una identidad sencilla pero fundamental:

$$d(\mathbf{u}, \mathbf{v}) = w(\mathbf{u} + \mathbf{v}) \quad (14)$$

En efecto, la suma módulo 2 produce un 1 exactamente en aquellas posiciones donde los vectores difieren. Puede verificarse que la distancia de Hamming es una métrica en sentido estricto: es no negativa, simétrica, se anula sólo si los vectores coinciden, y satisface la desigualdad triangular

$$d(\mathbf{u}, \mathbf{v}) + d(\mathbf{v}, \mathbf{w}) \geq d(\mathbf{u}, \mathbf{w}).$$

Esta última propiedad justifica la imagen geométrica que se usará todo el tiempo: las palabras de V_n son puntos de un espacio métrico —los vértices de un hipercubo de n dimensiones—, y las palabras de código son un subconjunto de puntos que conviene mantener lo más separados posible entre sí.

La *distancia mínima* del código C es la menor distancia entre dos palabras de código distintas:

$$d_{\min} = \min \{ d(\mathbf{v}, \mathbf{w}) : \mathbf{v}, \mathbf{w} \in C, \mathbf{v} \neq \mathbf{w} \}.$$

Es el parámetro que determina, por sí solo, las capacidades de detección y corrección del código, y por eso se acostumbra caracterizar un código de bloque por la terna $(n, k, d_{\min})$.

5.2. Distancia mínima y peso mínimo

Calcular $d_{\min}$ a partir de la definición exige comparar todos los pares de palabras de código, lo que son $\binom{2^k}{2}$ comparaciones: para $k = 239$ esto es tan impracticable como enumerar el código. La linealidad reduce el problema drásticamente.

Sea $w_{\min}$ el menor peso de las palabras de código *no nulas*. Entonces vale:

$$d_{\min} = w_{\min} \quad (15)$$

La demostración es breve y vale la pena hacerla en clase. Por (14), la distancia entre dos palabras $\mathbf{v}$ y $\mathbf{w}$ es el peso de $\mathbf{v} + \mathbf{w}$; y por linealidad, $\mathbf{v} + \mathbf{w}$ es a su vez una palabra de código, no nula porque $\mathbf{v} \neq \mathbf{w}$. Recíprocamente, el peso de cualquier palabra no nula $\mathbf{v}$ es su distancia a la palabra nula, que también pertenece al código. Los dos conjuntos de números sobre los que se toma el mínimo coinciden, y por lo tanto los mínimos coinciden.

El costo del cálculo pasa así de $\binom{2^k}{2}$ comparaciones a $2^k - 1$ evaluaciones de peso.

Sigue siendo exponencial, pero la reducción es enorme, y hay un camino mejor todavía.

5.3. Distancia mínima y columnas de H

El resultado más útil de esta sección vincula $d_{\min}$ con la estructura de la matriz de paridad. De (6) y (12), una palabra $\mathbf{v}$ de peso w pertenece al código si y sólo si las w columnas de H correspondientes a sus posiciones no nulas suman cero, es decir, si y sólo si son linealmente dependientes. Por lo tanto:

La distancia mínima de un código es igual a la menor cantidad de columnas de H que son linealmente dependientes.

De aquí se desprenden dos criterios de verificación inmediata que conviene tener siempre a mano:

- $d_{\min} \geq 2$ si y sólo si ninguna columna de H es nula.
- $d_{\min} \geq 3$ si y sólo si, además, todas las columnas de H son distintas entre sí.

Aplicando el segundo criterio a la matriz (8), cuyas siete columnas son los siete vectores no nulos de V_3 , se concluye que el código $(7, 4)$ tiene $d_{\min} \geq 3$; y como la Tabla 1 exhibe palabras de peso 3, resulta $d_{\min} = 3$ exactamente. Es la terna $(7, 4, 3)$.

Este criterio es, además, una herramienta de *diseño* y no sólo de análisis: construir un código con $d_{\min} \geq 3$ y $n - k$ bits de paridad se reduce a elegir n columnas distintas y no nulas de V_{n-k} , de las cuales hay $2^{n-k} - 1$ disponibles. La cota $n \leq 2^{n-k} - 1$ que se obtiene así es exactamente la que define a los códigos de Hamming como los más eficientes de su clase, tema de la unidad siguiente.

5.4. Distribución de pesos

Para el análisis de performance no alcanza con el peso mínimo: hace falta saber *cuántas* palabras hay de cada peso. Se define la *distribución de pesos* (*weight distribution*) del código como el conjunto de números $A_0, A_1, \dots, A_n$, donde A_i es la cantidad de palabras de código de peso i . Siempre vale $A_0 = 1$ (la palabra nula) y $\sum_i A_i = 2^k$. La información suele condensarse en el *polinomio enumerador de pesos*

$$A(z) = \sum_{i=0}^n A_i z^i.$$

Contando en la Tabla 1 se obtiene la distribución del código $(7, 4)$, que se muestra en la Tabla 2; su enumerador es

$$A(z) = 1 + 7z^3 + 7z^4 + z^7.$$

Obsérvese la simetría entre A_3 y A_4 y la presencia de la palabra de peso máximo 1111 111, ambas consecuencias de propiedades estructurales que se discutirán con los códigos de Hamming.

Tabla 2: Distribución de pesos del código (7, 4). El menor peso no nulo es 3, de donde $d_{\min} = 3$.

i	0	1	2	3	4	5	6	7
A_i	1	0	0	7	7	0	0	1

Determinar la distribución de pesos de un código arbitrario es, en general, un problema difícil. Existe no obstante una herramienta notable, la *identidad de MacWilliams*, que permite obtener el enumerador de pesos de un código a partir del de su código dual:

$$A(z) = 2^{-(n-k)}(1+z)^n B\left(\frac{1-z}{1+z}\right),$$

donde $B(z)$ es el enumerador del dual. Cuando $n - k$ es pequeño —es decir, cuando el dual tiene pocas palabras— esto convierte un problema intratable en un cálculo directo.

6. Capacidades de detección y corrección

6.1. Capacidad de detección

Un código detecta un patrón de error cuando la palabra recibida no es una palabra de código, esto es, cuando su síndrome es no nulo. Como el error pasa inadvertido únicamente si $\mathbf{e}$ es una palabra de código no nula, y como el peso mínimo de éstas es $d_{\min}$, ningún patrón de error de peso menor que $d_{\min}$ puede confundirse con una palabra válida. Por lo tanto:

Un código de distancia mínima $d_{\min}$ detecta todos los patrones de hasta $d_{\min}-1$ errores. (16)

La garantía es de peor caso, y como tal es pesimista: el código detecta *además* la inmensa mayoría de los patrones de peso mayor. De los $2^n - 1$ patrones de error no nulos, sólo $2^k - 1$ resultan indetectables, es decir, una fracción $2^{-(n-k)}$ del total. Para el (7, 4) eso significa 15 patrones indetectables sobre 127; para un código con 16 bits de paridad, apenas una parte en 65 536.

6.2. Capacidad de corrección

Corregir es más exigente que detectar: no basta con saber que hubo error, hay que identificar cuál palabra se transmitió. La condición geométrica es la siguiente. Trácese alrededor de cada palabra de código una “esfera” de radio t , entendida como el conjunto de todos los vectores a distancia de Hamming menor o igual que t de ella. Si esas esferas no se solapan, entonces toda palabra recibida con a lo sumo t errores cae dentro de la esfera de la palabra transmitida y de ninguna otra, y por lo tanto se la puede identificar sin ambigüedad.

La condición de no solapamiento es $2t < d_{\min}$, de donde

$$t = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor \quad (17)$$

Este t se denomina *capacidad de corrección* del código, y el código se dice t -corrector. Para el $(7, 4)$, con $d_{\min} = 3$, resulta $t = 1$: corrige todo error simple. La Figura 1 ilustra la construcción.

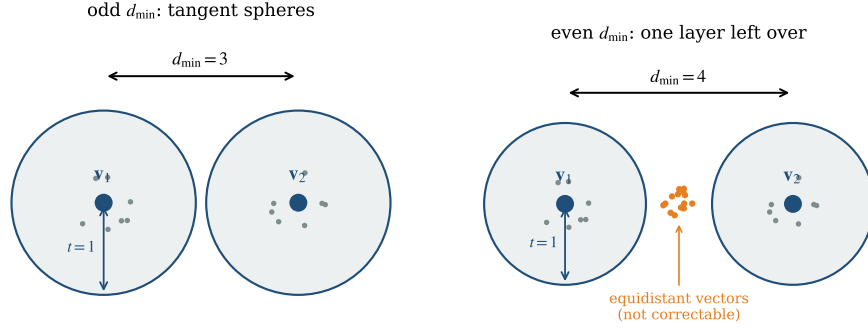


Figura 1: Interpretación geométrica de la distancia mínima. Cada palabra de código está rodeada de una esfera de radio t ; si las esferas no se solapan, todo patrón de hasta t errores es corregible. Cuando $d_{\min}$ es par sobra una capa de vectores equidistantes, que puede usarse para detección adicional. Figura generada con el script `fig1_spheres.py`.

De la figura se lee también el compromiso entre ambas capacidades. Un código de distancia mínima $d_{\min}$ puede operarse en varios modos:

- **Sólo detección:** detecta hasta $d_{\min} - 1$ errores.
- **Sólo corrección:** corrige hasta $t = \lfloor (d_{\min} - 1)/2 \rfloor$ errores.
- **Modo mixto:** corrige hasta λ errores y detecta hasta ℓ , con $\lambda < \ell$, siempre que $\lambda + \ell + 1 \leq d_{\min}$.

El modo mixto es el que se usa en la práctica cuando existe un canal de retorno: los errores leves se corrigen sobre la marcha y los severos se señalan para retransmisión, estrategia que se retomará en la unidad de esquemas híbridos FEC+ARQ.

6.3. Cotas sobre la distancia mínima

Conviene saber cuán lejos se puede llegar, es decir, qué $d_{\min}$ es alcanzable con $n - k$ bits de redundancia. Dos cotas elementales acompañan al curso entero.

La **cota de Singleton** se obtiene observando que, en forma sistemática, dos palabras cuyos mensajes difieren en un solo bit difieren en a lo sumo $1 + (n - k)$ posiciones:

$$d_{\min} \leq n - k + 1.$$

Los códigos que alcanzan la igualdad se denominan *de máxima distancia separable* (MDS); los Reed–Solomon de la Unidad 8 son el ejemplo por excelencia, y ésta es la razón de su eficiencia.

La **cota de Hamming**, o cota de empaquetamiento de esferas, se obtiene contando: las 2^k esferas de radio t deben caber, sin solaparse, dentro de los 2^n vectores de V_n . Como cada esfera contiene $\sum_{i=0}^t \binom{n}{i}$ vectores,

$$2^{n-k} \geq \sum_{i=0}^t \binom{n}{i} \quad (18)$$

Los códigos que alcanzan la igualdad se denominan *perfectos*: sus esferas cubren V_n sin dejar ningún vector afuera, de modo que todo vector recibido está a distancia $\leq t$ de exactamente una palabra de código. El $(7, 4)$ lo es, puesto que $2^3 = 8 = 1 + 7$. Los códigos perfectos son notablemente escasos: además de los de repetición de longitud impar, sólo existen los Hamming y el Golay $(23, 12)$, ambos objeto de la unidad siguiente.

7. Decodificación

7.1. El criterio de máxima verosimilitud sobre el BSC

Se retoma ahora la ambigüedad planteada en (13): dado un síndrome, hay 2^k patrones de error compatibles, y hay que elegir uno. El criterio racional es el de *máxima verosimilitud* (*maximum likelihood*): elegir el patrón más probable a la luz del modelo de canal.

Sobre un canal binario simétrico con probabilidad de transición $p < 1/2$, la probabilidad de que ocurra un patrón de error específico de peso w es

$$P(\mathbf{e}) = p^w (1 - p)^{n-w}.$$

Como $p < 1/2$ implica $p/(1 - p) < 1$, esta expresión es *monótonamente decreciente* en w : cuantos más errores, menos probable el patrón. Por lo tanto, entre los 2^k candidatos compatibles con el síndrome observado, el más probable es simplemente el de **menor peso**. Equivalentemente, en términos de (14), se elige la palabra de código *más cercana* a la recibida: la decodificación por máxima verosimilitud sobre el BSC coincide con la decodificación por mínima distancia de Hamming.

Aquí es importante fijar el estatus de este criterio. Es *óptimo* —ninguna regla de decisión puede tener menor probabilidad de error—, pero **no es infalible**: si el canal introdujo un patrón de peso alto, el decodificador elegirá otro de peso bajo y entregará una palabra equivocada. La cantidad de errores que se garantiza corregir es t ; más allá de eso, el decodificador acierta a veces y falla otras.

7.2. Standard array

El *standard array* es la materialización exhaustiva del criterio anterior. Se trata de un arreglo de 2^{n-k} filas por 2^k columnas que clasifica los 2^n vectores de V_n , y se construye así:

1. En la primera fila se listan las 2^k palabras de código, empezando por la palabra nula.
2. Para cada fila siguiente, se elige un vector de V_n todavía no listado, con el **menor peso posible** entre los disponibles, y se lo coloca en la primera columna. Se

completa la fila sumando ese vector a cada una de las palabras de código de la primera fila.

3. Se repite hasta agotar los 2^n vectores.

El vector que encabeza cada fila se denomina *líder de coset* (*coset leader*), y la fila completa es un *coset* del código. Puede demostrarse que el arreglo así construido tiene tres propiedades: todo vector de V_n aparece **exactamente una vez**; dos vectores de una misma fila difieren en una palabra de código; y **todos los vectores de una misma fila tienen el mismo síndrome**, distinto del de las demás filas. Esta última es la que conecta el standard array con lo visto antes: las filas están en correspondencia biunívoca con los 2^{n-k} síndromes posibles.

La regla de decodificación es entonces inmediata. Localizado $\mathbf{r}$ en el arreglo, se toma como patrón de error estimado el líder de su fila, y como palabra decodificada el encabezado de su columna. Como los líderes se eligieron de peso mínimo, esta regla implementa exactamente la decodificación por máxima verosimilitud.

Dos consecuencias merecen destacarse:

- Un patrón de error se corrige correctamente **si y sólo si** es un líder de coset. Hay entonces exactamente 2^{n-k} patrones corregibles, uno por fila, y ni uno más: es un límite duro impuesto por la redundancia disponible.
- Cuando existen varios vectores del mismo peso mínimo dentro de un coset, la elección del líder es arbitraria y el decodificador es genuinamente ambiguo. Esto ocurre siempre que el peso del líder excede t , y es la manifestación concreta del solapamiento de esferas de la Figura 1.

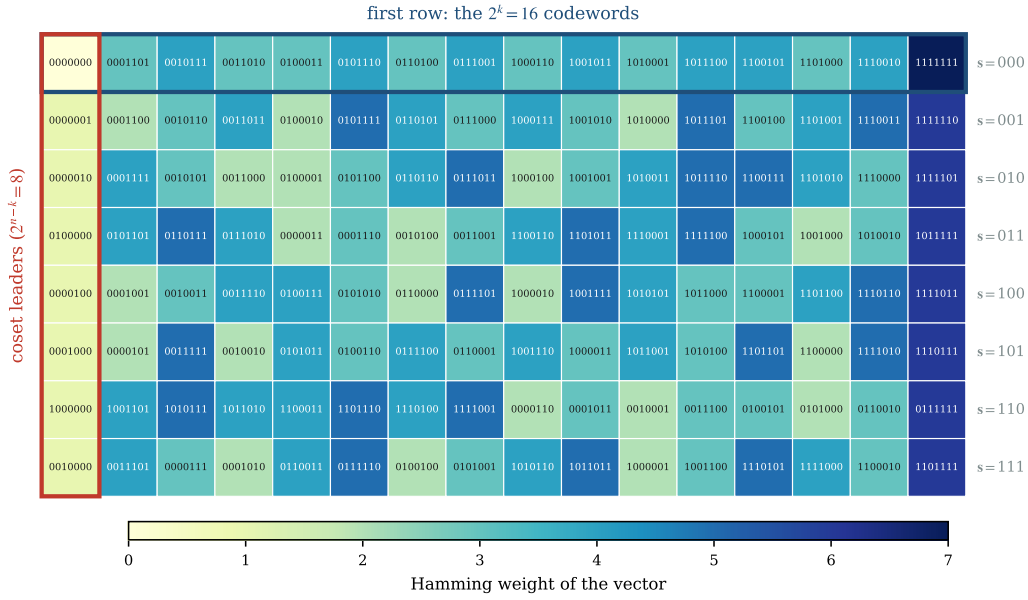


Figura 2: Standard array completo del código $(7, 4)$. Las 8 filas son los cosets, una por síndrome, y las 16 columnas están encabezadas por las palabras de código; los 128 vectores de V_7 aparecen exactamente una vez. El color indica el peso de Hamming, lo que hace visible que la primera fila y la primera columna concentran los vectores de menor peso. Decodificar consiste en proyectar sobre el encabezado de la columna. Figura generada con el script `fig4_standard_array.py`.

El standard array es una herramienta conceptual excelente y una herramienta práctica pésima: para el $(7, 4)$ tiene 128 entradas, todavía dibujable en el pizarrón, pero para el $(255, 239)$ tendría 2^{255} . Nunca se implementa; se lo estudia porque explica qué es lo que un decodificador óptimo hace, y porque da el marco para contar exactamente cuántos patrones son corregibles.

7.3. Decodificación por síndrome

La observación de que todos los vectores de una fila comparten el mismo síndrome permite comprimir el standard array quedándose únicamente con su primera columna. El resultado es la *decodificación por síndrome*, que consta de tres pasos:

1. Calcular el síndrome $\mathbf{s} = \mathbf{r}H^T$.
2. Buscar en una tabla el líder de coset $\mathbf{e}$ asociado a ese síndrome.
3. Entregar $\hat{\mathbf{v}} = \mathbf{r} + \mathbf{e}$.

La tabla tiene 2^{n-k} entradas en lugar de 2^n , lo que representa una reducción por un factor 2^k . Para el $(7, 4)$ se pasa de 128 a 8 entradas, listadas en la Tabla 3. Obsérvese que, salvo el síndrome nulo, cada uno de los siete síndromes restantes corresponde a un error simple, y que el síndrome coincide con la columna de H de la posición errónea, tal como anticipaba (12). Esto último no es general: es una particularidad de los códigos perfectos con $t = 1$.

Tabla 3: Tabla de decodificación por síndrome del código (7, 4). Los ocho síndromes agotan los $2^{n-k} = 8$ cosets, y los líderes son la palabra nula y los siete errores simples.

Síndrome s	Líder de coset e	Interpretación
000	0000 000	sin errores
110	1000 000	error en la posición 0
011	0100 000	error en la posición 1
111	0010 000	error en la posición 2
101	0001 000	error en la posición 3
100	0000 100	error en la posición 4
010	0000 010	error en la posición 5
001	0000 001	error en la posición 6

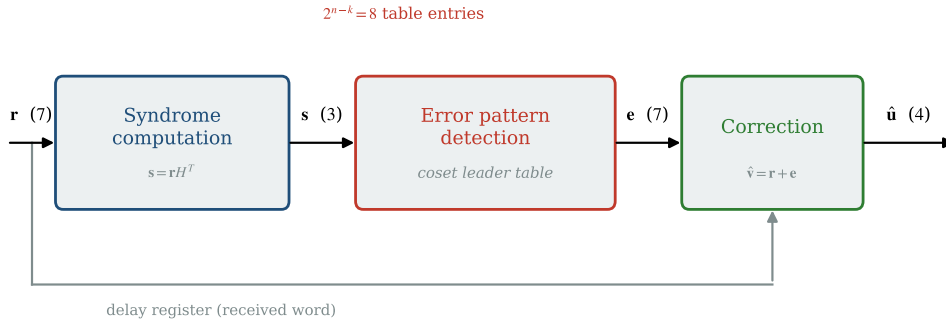


Figura 3: Diagrama en bloques de un decodificador por síndrome. El bloque de cálculo del síndrome es una red de compuertas XOR determinada por H ; el bloque de detección de patrón es, en el caso del (7, 4), un decodificador de 3 a 7 líneas; y la corrección se realiza con n compuertas XOR. Figura generada con el script `fig2_decoder.py`.

El costo sigue siendo exponencial en la redundancia, y por eso este esquema tampoco escala: para el (255, 239) la tabla tendría $2^{16} = 65\,536$ entradas, todavía manejable, pero para redundancias mayores deja de serlo. La salida a este problema es imponer *más* estructura al código —la estructura cíclica de la Unidad 6— de modo que el patrón de error pueda calcularse algebraicamente a partir del síndrome, en lugar de buscarse en una tabla. Los algoritmos de Berlekamp–Massey, Chien y Forney de las Unidades 7 y 8 son exactamente eso: una tabla de síndromes reemplazada por aritmética en $GF(2^m)$.

7.4. Decodificación incompleta

Una variante que aparece en muchos sistemas reales es la *decodificación incompleta* (o *bounded-distance decoding*): en lugar de asignar un líder a cada uno de los 2^{n-k} síndromes, se corrigen únicamente los patrones de peso $\leq t$ y se declara *falla de decodificación* para todos los demás. Se renuncia así a corregir algunos patrones que

el esquema completo sí corregiría, a cambio de dos ventajas: la tabla se achica, y sobre todo el decodificador nunca entrega en silencio una palabra equivocada por un error de peso alto, sino que levanta una bandera. En sistemas con retransmisión, esa bandera es más valiosa que la corrección adicional.

8. Performance sobre el canal binario simétrico

8.1. Probabilidad de error no detectado

Considérese el código operando en modo de sola detección sobre un BSC con probabilidad de transición p . Como se estableció, el error pasa inadvertido exactamente cuando el patrón de error coincide con una palabra de código no nula. Sumando las probabilidades de esos $2^k - 1$ patrones, y agrupándolos por peso mediante la distribución $\{A_i\}$, se obtiene

$$P_u(E) = \sum_{i=1}^n A_i p^i (1-p)^{n-i} \quad (19)$$

Ésta es la razón por la cual la distribución de pesos importa y no alcanza con $d_{\min}$. Para el código (7, 4), con el enumerador calculado antes,

$$P_u(E) = 7p^3(1-p)^4 + 7p^4(1-p)^3 + p^7.$$

Con $p = 10^{-2}$ resulta $P_u(E) \approx 6,8 \cdot 10^{-6}$: de cada millón de bloques con errores, unos pocos pasan inadvertidos. Nótese que el término dominante para p pequeño es el asociado a $d_{\min}$, de modo que $P_u(E) \approx A_{d_{\min}} p^{d_{\min}}$; la distancia mínima gobierna el exponente y la multiplicidad $A_{d_{\min}}$ gobierna el coeficiente.

Existe además un resultado general reconfortante: para todo n y k existen códigos (n, k) cuya probabilidad de error no detectado satisface

$$P_u(E) \leq 2^{-(n-k)} [1 - (1-p)^n] \leq 2^{-(n-k)},$$

es decir, decrece exponencialmente con la cantidad de bits de paridad. Agregar un bit de paridad divide la cota por dos. Esta es la justificación cuantitativa del uso de CRC de 32 bits en sistemas de comunicaciones, incluido el módulo “CRC-32 calculator and checker” del trabajo práctico final: la probabilidad de que un error pase inadvertido es del orden de $2^{-32} \approx 2 \cdot 10^{-10}$.

Conviene advertir que la desigualdad anterior afirma que *existen* buenos códigos, no que todos lo sean. Hay códigos (n, k) cuya $P_u(E)$ se comporta mal, e incluso crece con p , de modo que la elección concreta del código no es indiferente.

8.2. Probabilidad de error de decodificación

Operando en modo de corrección con un decodificador completo, la decodificación es correcta si y sólo si el patrón de error es un líder de coset. Denotando por α_i la cantidad de líderes de coset de peso i , la probabilidad de decodificación errónea es

$$P(E) = 1 - \sum_{i=0}^n \alpha_i p^i (1-p)^{n-i} \quad (20)$$

Para un código perfecto t -corrector la expresión se simplifica notablemente, porque los líderes son exactamente todos los patrones de peso $\leq t$, esto es, $\alpha_i = \binom{n}{i}$ para $i \leq t$ y $\alpha_i = 0$ en otro caso:

$$P(E) = 1 - \sum_{i=0}^t \binom{n}{i} p^i (1-p)^{n-i} = \sum_{i=t+1}^n \binom{n}{i} p^i (1-p)^{n-i}.$$

Esta última fórmula, válida para códigos perfectos y muy buena aproximación para los demás, es la que se usará en el TP1 como referencia analítica contra la cual contrastar la simulación.

8.3. Ganancia de codificación

La probabilidad de error por bloque no es la métrica de interés en un enlace real; lo es la tasa de error de bit *después* de decodificar, y en función de la relación señal a ruido por bit de información E_b/N_0 . El eslabón que falta es la relación entre p y E_b/N_0 : para modulación BPSK sobre canal AWGN,

$$p = Q\left(\sqrt{\frac{2RE_b}{N_0}}\right),$$

donde el factor $R = k/n$ refleja que, a igual potencia y tasa de información, la energía disponible por *símbolo transmitido* se reduce porque hay que transmitir más símbolos.

Ese factor R es la clave para entender por qué codificar no siempre conviene. A baja relación señal a ruido, la penalización por reducir la energía por símbolo supera la mejora que aporta la corrección de errores, y el sistema codificado resulta *peor* que el no codificado; el punto donde ambas curvas se cruzan es el *umbral de codificación*. Por encima de él, la distancia horizontal entre las curvas es la *ganancia de codificación* (*coding gain*), que se mide en dB y expresa cuánta potencia de transmisión se ahorra gracias al código. La Figura 4 muestra ambas curvas para el código (7,4).

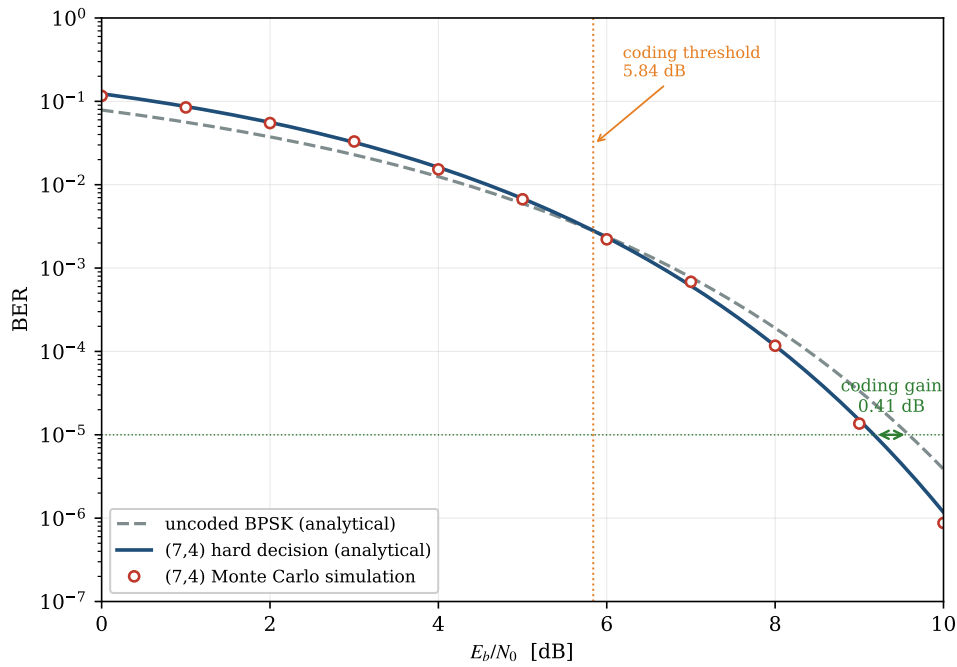


Figura 4: Tasa de error de bit versus E_b/N_0 para BPSK sobre canal AWGN, sin codificar y con el código (7, 4) con decodificación por síndrome. Se indican el umbral de codificación y la ganancia de codificación a $\text{BER} = 10^{-5}$. Figura generada con el script `fig3_ber.py`.

Esta figura es, esencialmente, el entregable del TP1: el modelo Python debe reproducirla, y la implementación en Verilog debe producir exactamente las mismas decisiones que ese modelo sobre los mismos vectores de entrada.

9. Casos particulares

Se cierra el recorrido con tres familias que, por su simplicidad, permiten ver los conceptos anteriores en estado puro, y que además aparecen como componentes dentro de sistemas mayores.

9.1. Códigos de paridad simple

El *código de paridad simple* (*single-parity-check code*) $(n, n-1)$ agrega a los $n-1$ bits de mensaje un único bit de paridad, elegido de modo que la cantidad total de unos sea par. Su matriz de paridad es una única fila de unos,

$$H = \begin{bmatrix} 1 & 1 & \cdots & 1 \end{bmatrix},$$

y su síndrome es un solo bit: la paridad total de la palabra recibida.

Todas sus palabras tienen peso par, de donde $d_{\min} = 2$. Aplicando (16) y (17): detecta todo patrón con una cantidad *impar* de errores, y no corrige nada. Su tasa $R = (n-1)/n$ es casi 1, de modo que su costo en ancho de banda es despreciable. Es el código de máxima eficiencia y mínima protección, y por eso aparece en todas partes donde hace falta una verificación barata.

9.2. Códigos de repetición

En el extremo opuesto, el *código de repetición* (*repetition code*) $(n, 1)$ tiene exactamente dos palabras: la nula y la de todos unos. Su matriz generadora es una única fila de unos —es decir, es el *dual* del código de paridad simple, y viceversa—, y su distancia mínima es $d_{\min} = n$, la máxima posible.

Para n impar corrige hasta $t = (n - 1)/2$ errores, y la decodificación por mínima distancia se reduce a una **votación por mayoría**. Su tasa $R = 1/n$ es pésima, y por eso rara vez se usa como código en sí mismo; aparece, en cambio, como componente interno de esquemas mayores y como caso límite útil para razonar. Resulta además, para n impar, un código perfecto: se verifica sin dificultad que satisface con igualdad la cota (18).

El par formado por estos dos códigos ilustra el compromiso fundamental de toda la disciplina. Fijado n , la paridad simple maximiza la tasa y la repetición maximiza la distancia mínima; todos los códigos interesantes viven en algún punto intermedio, y el arte del diseño consiste en acercarse lo más posible a las cotas de la sección anterior.

9.3. Códigos auto-duales

Un código C es *auto-dual* (*self-dual*) si coincide con su propio código dual, $C = C_d$. Como $\dim(C) = k$ y $\dim(C_d) = n - k$, esto obliga a $k = n/2$, de modo que un código auto-dual tiene necesariamente longitud par y tasa exactamente $R = 1/2$. Sus matrices G y H generan el mismo espacio y, en particular, puede tomarse $H = G$.

La existencia de estos códigos depende de una propiedad de $GF(2)$ que se señaló en la unidad anterior y que no tiene análogo sobre los reales: un vector no nulo puede ser ortogonal a sí mismo. En efecto, $C \subseteq C_d$ exige que toda palabra de código sea ortogonal a todas las demás y a sí misma, es decir, $\mathbf{v} \cdot \mathbf{v} = w(\mathbf{v}) = 0$, lo que se traduce en que **todas las palabras de un código auto-dual tienen peso par**.

El ejemplo mínimo es el código de repetición $(2, 1)$, cuyas palabras son 00 y 11. El ejemplo relevante es el código de Hamming *extendido* $(8, 4)$, que se obtiene agregando un bit de paridad global al $(7, 4)$ estudiado en esta unidad: resulta auto-dual, con $d_{\min} = 4$, y por lo tanto capaz de corregir un error y detectar dos simultáneamente. Se lo retomará en la unidad siguiente al tratar los códigos SEC-DED.

10. Cierre: qué se hace con todo esto

Esta unidad estableció el vocabulario con el que se describirán todos los códigos de bloque del curso. Cada concepto tiene un destinatario preciso:

- Las matrices G y H y la forma sistemática son la descripción general de todo código de bloque lineal. Las familias de la Unidad 5 —Hamming, Reed–Muller, Golay— son elecciones particulares de esas matrices, y el criterio de las columnas de H desarrollado aquí es lo que permite construirlas y calcular su $d_{\min}$ sin enumerar sus palabras.
- El **síndrome** es el objeto central de toda la decodificación algebraica que sigue. En la Unidad 6 dejará de calcularse con un producto matricial para calcularse como resto de una división polinómica, con un registro de desplazamiento; en las Unidades 7 y 8 se lo evaluará en potencias de α sobre $GF(2^m)$, y de ahí sal-

drán directamente los módulos “BCH(255,239) syndrome calculator” del trabajo práctico final.

- La relación entre **distancia mínima** y capacidades de detección y corrección, junto con las cotas de Singleton y de Hamming, es el marco en el que se comparan todos los códigos del curso. Los Reed–Solomon serán interesantes por alcanzar la cota de Singleton; los turbo y LDPC, por acercarse al límite de Shannon con distancias mínimas modestas pero excelentes distribuciones de pesos.
- La **decodificación por síndrome** y su costo exponencial en $n - k$ son la motivación de todo lo que sigue. Cada estructura adicional que se imponga al código —ciclicidad, raíces prescriptas, dispersión de H — tendrá como propósito reemplazar la tabla de líderes de coset por un algoritmo.
- La expresión (20) y la noción de **ganancia de codificación** son las métricas con las que se evaluará cada código, y las que el modelo Python del TP1 debe reproducir para servir de referencia a la implementación RTL.

En síntesis, la Unidad 3 aportó la aritmética y esta unidad aportó la estructura. Lo que resta del curso consiste en descubrir qué estructura adicional conviene imponer para que la decodificación deje de ser una búsqueda en tabla y se vuelva un cálculo.

Referencias

- [1] S. Lin y D. J. Costello, *Error Control Coding*, 2^a ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2004. (Capítulo 3: *Linear Block Codes*.)
- [2] E. Sanvicente, *Understanding Error Control Coding*. Springer Nature Switzerland AG, 2019. doi: 10.1007/978-3-030-05840-1. (Capítulo 2: *A First Look at Block Coders*.)
- [3] F. J. MacWilliams y N. J. A. Sloane, *The Theory of Error-Correcting Codes*. Amsterdam: North-Holland, 1977. (Referencia clásica para la identidad de MacWilliams y las cotas sobre $d_{\min}$.)